
SOFTWARE ENGINEERING

NOTEBOOK

Table of Contents

Table of Contents	2
1 Algorithms	5
1 Time Complexity	5
1.1 Big O notation	6
1.2 Omega notation	6
1.3 Theta notation	6
1.4 Meaning of previous notations - Insertion sort case	6
1.5 Running times of well known algorithms	8
1.6 Methods for solving the recurrence	9
2 Space Complexity	9
Algorithms Bibliography	10
2 Solid Principles	11
1 Single-Responsibility	12
2 Open-Close	12
3 Liskov Substitution	12
4 Interface Segregation	12
5 Dependency Inversion	12
Solid Principles Bibliography	13
3 Idioms	15
1 C++ Idioms	15
1.1 PImpl Idiom	15
1.2 Fast PImpl Idiom	16
1.3 Handle/Object Idiom	16
1.4 Copy and Swap Idiom	16
1.5 RAII - Resource Acquisition is Initialization	16
1.6 DBDII - Dynamic Binding During Initialization Idiom	16
1.7 Named Constructor Idiom	16
1.8 Construct On First Use Idiom	16
1.9 Nifty Counter Idiom	16

1.10	Named Parameter Idiom	17
1.11	Virtual Friend Function Idiom	17
1.12	CRTP - Curiously recurring template pattern	17
1.13	Type Erasure	17
	Idioms Bibliography	18
4	Containers	19
1	Links to the CPP Reference documentation	21

Algorithms

1 Time Complexity

Time complexity for an algorithm has three different facets

- The *worst-case* complexity of the algorithm is the function defined by the maximum number of steps taken in any instance of size n .
- The *best-case* complexity of the algorithm is the function defined by the minimum number of steps taken in any instance of size n .
- The *average-case* complexity or expected time of the algorithm, which is the function defined by the average number of steps over all instances of size n .

1.1 Big O notation

$$f(n) = O(g(n)) \quad \text{if} \quad 0 \leq f(n) \leq cg(n) \forall n \geq n_0 \quad (1.1)$$

For all values $n \geq n_0$, $f(n)$ is on or below $cg(n)$.

A constant multiple of $g(n)$ is an *asymptotic upper bound* for $f(n)$.

1.2 Omega notation

$$f(n) = \Omega(g(n)) \quad \text{if} \quad 0 \leq cg(n) \leq f(n) \forall n \geq n_0 \quad (1.2)$$

For all values $n \geq n_0$, $f(n)$ is on or above $cg(n)$.

A constant multiple of $g(n)$ is an *asymptotic lower bound* for $f(n)$.

1.3 Theta notation

$$f(n) = \Theta(g(n)) \quad \text{if} \quad 0 \leq c_1g(n) \leq f(n) \leq c_2g(n) \forall n \geq n_0 \quad (1.3)$$

For all values $n \geq n_0$, $f(n)$ is equal to $g(n)$ within a constant factor.

$g(n)$ is an *asymptotically tight bound* for $f(n)$.

1.4 Meaning of previous notations - Insertion sort case

Big O notation

describes an *upper bound*. When we use it to *bound the worst-case running time* of an algorithm, we have a bound on the running time of the algorithm on *every input*.

Thus, the $O(n^2)$ bound on worst-case running time of *insertion sort* also applies to its running time on every input.

Omega notation

describes a *lower bound*. When we say that the running time (no modifier) of an algorithm is $\Omega(g(n))$, we mean that *no matter what particular input of size n is chosen* for each value of n , the running time on that input is at least a constant times $g(n)$, for sufficiently large n .

Equivalently, we are giving a *lower bound on the best-case running time* of an algorithm. For example, the best-case running time of insertion sort is $\Omega(n)$, which implies that the running time of insertion sort is $\Omega(n)$.

The *running time of insertion sort therefore belongs to both $\Omega(n)$ and $O(n^2)$* , since it falls anywhere between a linear function of n and a quadratic function of n . Moreover, these bounds are asymptotically as tight as possible: for instance, the running time of insertion sort is not $\Omega(n^2)$, since there exists an input for which insertion sort runs in $\Theta(n)$ time (e.g., when the input is already sorted). It is not contradictory, however, to say that the worst-case running time of insertion sort is $\Omega(n^2)$, since there exists an input that causes the algorithm to take $\Omega(n^2)$ time.

Theta notation

describes a *tight bound*. The $\Theta(n^2)$ notation bound on the worst-case running time of insertion sort, however, does not imply a $\Theta(n^2)$ bound on the running time of insertion

sort on every input. For example, when the input is already sorted, insertion sort runs in $\Theta(n)$ time. Technically, it is an abuse to say that the running time of insertion sort is $O(n^2)$, since for a given n , the actual running time varies, depending on the particular input of size n . When we say "the running time is $O(n^2)$ ", we mean that there is a function $f(n)$ that is $O(n^2)$ such that for any value of n , no matter what particular input of size n is chosen, the running time on that input is bounded from above by the value $f(n)$. Equivalently, we mean that the worst-case running time is $O(n^2)$.

1.5 Running times of well known algorithms

Algorithm	Best	Average	Worst	Worst Space Complexity
Quicksort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Mergesort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Timsort	$O(n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Heapsort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Shell Sort	$O(n)$	$O((n \log n)^2)$	$O((n \log n)^2)$	$O(1)$
Bucket Sort	$O(n + k)$	$O(n + k)$	$O(n^2)$	$O(n)$
Radix Sort	$O(nk)$	$O(nk)$	$O(nk)$	$O(n + k)$

Verbal description of the previous sorting algorithms:

- **Quicksort**

Efficient, general-purpose sorting algorithm, **slightly faster** than **merge sort** and **heapsort** for **randomized data**, particularly on larger distributions.

It selects a **pivot element** from the array and partitions the other elements into **two sub-arrays**, according to whether they are **less than or greater than the pivot**.

For this reason, it is sometimes called **partition-exchange sort**.

The sub-arrays are then **sorted recursively**.

- **Mergesort**

First, divide the list into the smallest unit (1 element), then **compare each element with the adjacent** list to sort and merge the two adjacent lists.

Finally, all the elements are sorted and merged.

- **Timsort**

a hybrid, stable sorting algorithm, **derived from merge sort and insertion sort**, designed to **perform well on many kinds of real-world data**. It was implemented by Tim Peters in 2002 for use in the Python programming language. The algorithm finds **subsequences of the data that are already ordered** (runs) and uses them to **sort the remainder more efficiently**. This is done by merging runs until certain criteria are fulfilled. Timsort was Python's standard sorting algorithm from version 2.3 to version 3.10

- **Heapsort**

comparison-based sorting algorithm which can be thought of as

an implementation of selection sort using the right data structure. Like selection sort, heapsort divides its input into a sorted and an unsorted region, and it iteratively shrinks the unsorted region by extracting the largest element from it and inserting it into the sorted region. Unlike selection sort, heapsort does not waste

time with a linear-time scan of the unsorted region; rather, heap sort maintains the unsorted region in a heap data structure to efficiently find the largest element in each step.

- **Bubble Sort**

a simple sorting algorithm that repeatedly steps through the input list element by element, comparing the current element with the one after it, swapping their values if needed. These passes through the list are repeated until no swaps have to be performed during a pass, meaning that the list has become fully sorted

- **Insertion Sort**

iterates, consuming one input element each repetition, and grows a sorted output list. At each iteration, insertion sort removes one element from the input data, finds the location it belongs within the sorted list, and inserts it there. It repeats until no input elements remain.

- **Selection Sort**

repeatedly identifies the smallest remaining unsorted element and puts it at the end of the sorted portion of the array

- **Shell Sort**

- **Bucket Sort**

Each bucket is then sorted individually, either using a different sorting algorithm, or by recursively applying the bucket sorting algorithm. It is a distribution sort, a generalization of pigeonhole sort that allows multiple keys per bucket, and is a cousin of radix sort in the most-to-least significant digit flavor. Bucket sort can be implemented with comparisons and therefore can also be considered a comparison sort algorithm. The computational complexity depends on the algorithm used to sort each bucket, the number of buckets to use, and whether the input is uniformly distributed.

- **Radix Sort**

1.6 Methods for solving the recurrence

There are three methods for solving recurrences:

- the substitution method
- the recursion-tree method
- the master theorem

2 Space Complexity

Algorithms Bibliography

- [90] Robert Sedgewick and Kevin Wayne. *Algorithms*. Addison-Wesley.
- [91] Steven S. Skiena. *The Algorithm Design Manual*. Springer.
- [92] *Quicksort*. [Wikipedia](#).
- [93] *Merge Sort*. [Wikipedia](#).
- [94] *Timsort*. [Wikipedia](#).
- [95] *Heapsort*. [Wikipedia](#).
- [96] *Bubble Sort*. [Wikipedia](#).
- [97] *Insertion Sort*. [Wikipedia](#).
- [98] *Selection Sort*. [Wikipedia](#).
- [99] *Shell Sort*. [Wikipedia](#).
- [100] *Bucket Sort*. [Wikipedia](#).
- [101] *Radix Sort*. [Wikipedia](#).

Solid Principles

SOLID is a mnemonic acronym for five design principles intended to make object-oriented designs more understandable, flexible, and maintainable.

The principles are a subset of many principles promoted by American software engineer and instructor **Robert C. Martin**, [75]- [76]- [77].

They were first introduced in his 2000 paper Design Principles and Design Patterns, while the acronym was introduced later, around 2004, by **Michael Feathers** [85].

Although the SOLID principles apply to any object-oriented design, they can also form a core philosophy for methodologies such as agile development or adaptive software development.

1 Single-Responsibility

"There should never be more than one reason for a class to change." [79]

In other words, every class should have only one responsibility. [80]

2 Open-Close

"Software entities ... should be open for extension, but closed for modification." [81]

3 Liskov Substitution

"Functions that use pointers or references to base classes must be able to use objects of derived classes without knowing it." [82]

See also: [Design by contract](#).

4 Interface Segregation

"Clients should not be forced to depend upon interfaces that they do not use." [83] [78]

5 Dependency Inversion

"Depend upon abstractions, [not] concretions." [84] [78]

Solid Principles Bibliography

- [75] Robert C. Martin. *Principles of OOD*. [WayBack Machine](#). 2014.
- [76] Robert C. Martin. *Getting a SOLID start*. [Uncle Bob Consulting LLC](#). 2009.
- [77] Sandu Metz. *SOLID Object-Oriented*. [Ghost Archive - Gotham Ruby Conference](#). 2009.
- [78] Robert C. Martin. *Design Principles and Design Patterns*. [Internet Archive - Objectmentor](#). 2000.
- [79] Robert C. Martin. *Single Responsibility Principle*. [Internet Archive - Objectmentor](#). 2015.
- [80] Robert C. Martin. *Agile Software Development, Principles, Patterns, and Practices*. [Internet Archive - Objectmentor](#). 2003.
- [81] *Open/Closed Principle*. [Internet Archive - Objectmentor](#).
- [82] *Liskov Substitution Principle*. [Internet Archive - Objectmentor](#).
- [83] *Interface Segregation Principle*. [Internet Archive - Objectmentor](#).
- [84] *Dependency Inversion Principle*. [Internet Archive - Objectmentor](#).
- [85] *Clean Architecture: A Craftman's Guide to Software Structure and Design*.
- [86] *Clean Architecture: A Craftman's Guide to Software Structure and Design*. [here](#).

3

CHAPTER

Idioms

1 C++ Idioms

An idiom can be considered as the most natural way to express something or to solve a problem. More specifically, an idiom is a group of code fragments sharing an equivalent semantic role, which recurs frequently in one or more programming languages or libraries.

A comprehensive list of C++ Idioms can be found at [\[124\]](#)

1.1 PImpl Idiom

[c++reference documentation about the PImpl Idiom.](#)

"Pointer to implementation" or "pImpl" is a C++ programming technique that **removes implementation details of a class from its object representation** by placing them in a separate class, accessed **through an opaque pointer**.

This idiom is also known as **Compiler Firewall Idiom**.

Problematic Scenario

Whenever file class definition changes in a header, all users of that class must be recompiled. The reason is the textual inclusion on which the C++ build model is based. The user is assumed to know two features which could be affected by private members: **size** and **layout** and **functions**.

Solution

```
1 // Pimpl idiom - basic idea
2 class widget {
3     // :::
4 private:
5     struct impl;           // things to be hidden go here
6     impl* pimpl_;         // opaque pointer to forward-declared class
7 };
```

1.2 Fast PImpl Idiom

1.3 Handle/Object Idiom

1.4 Copy and Swap Idiom

This idiom aims at creating a strong exception-safe implementation of the overloaded assignment operator.

This idiom makes use of the **RAII** idiom in order to acquire the new resource before it forfeits its current resource. If the acquisition is successful

1.5 RAII - Resource Acquisition is Initialization

1.6 DBDII - Dynamic Binding During Initialization Idiom

[calling-virtuals-from-ctor-idiom](#)

1.7 Named Constructor Idiom

[135]

1.8 Construct On First Use Idiom

[136]

1.9 Nifty Counter Idiom

[137]

1.10 Named Parameter Idiom

[\[139\]](#)

1.11 Virtual Friend Function Idiom

[\[140\]](#)

1.12 CRTP - Curiously recurring template pattern

1.13 Type Erasure

Idioms Bibliography

- [124] *More C++ Idioms*. [Wikibooks](#).
- [125] *Pimpl Idiom*. [Wiki Wiki Web](#).
- [126] *CRTP - Curiously Recurring Template Pattern*. [Wiki Wiki Web](#).
- [127] *CRTP - Curiously Recurring Template Pattern*. [Cppreference](#).
- [128] Herb Sutter. *The Fast Pimpl Idiom*. [Guru of the Week - 28](#).
- [129] Herb Sutter. *Compilation Firewalls*. [Guru of the Week - 24](#).
- [130] Herb Sutter. *Compilation Firewalls (C++11-updated version of GotW #24)*. [Guru of the Week - 100](#).
- [131] Herb Sutter. *The Joy of Pimpls (or more about the Compiler-Firewall Idiom)*. [More About Compiler Firewalls](#).
- [132] *What is the copy and swap idiom?* [Stackoverflow](#).
- [133] Herb Sutter. *Exception-Safe Class Design, Part I: Copy Assignment*. [Guru of the Week - 59](#).
- [134] Herb Sutter. *Exception-Safe Class Design, Part II: Inheritance*. [Guru of the Week - 60](#).
- [135] *What is the Named Constructor Idiom?* [ISOC++ Wiki](#).
- [136] *Construct On First Use Idiom*. [ISOC++ Wiki](#).
- [137] *Nifty Counter Idiom*. [ISOC++ Wiki](#).
- [138] *Nifty Counter Idiom*. [Wikibook](#).
- [139] *Named Parameter Idiom*. [ISOC++ Wiki](#).
- [140] *Virtual Friend Function Idiom*. [ISOC++ Wiki](#).
- [141] *C++ Core Guidelines*. [Standard Cpp Foundation Github](#).

4

CHAPTER

Containers

The Containers library is a *generic collection of class templates and algorithms* that allow programmers *to easily implement common data structures* like queues, lists and stacks. The container classes are:

- *sequence containers*
implement data structures which can be accessed sequentially.
- *associative containers*,
can be searched in $O(\log n)$.
- *unordered associative containers*
can be searched in $O(1)$ on average and in $O(n)$ in the worst one.

Decision Charts

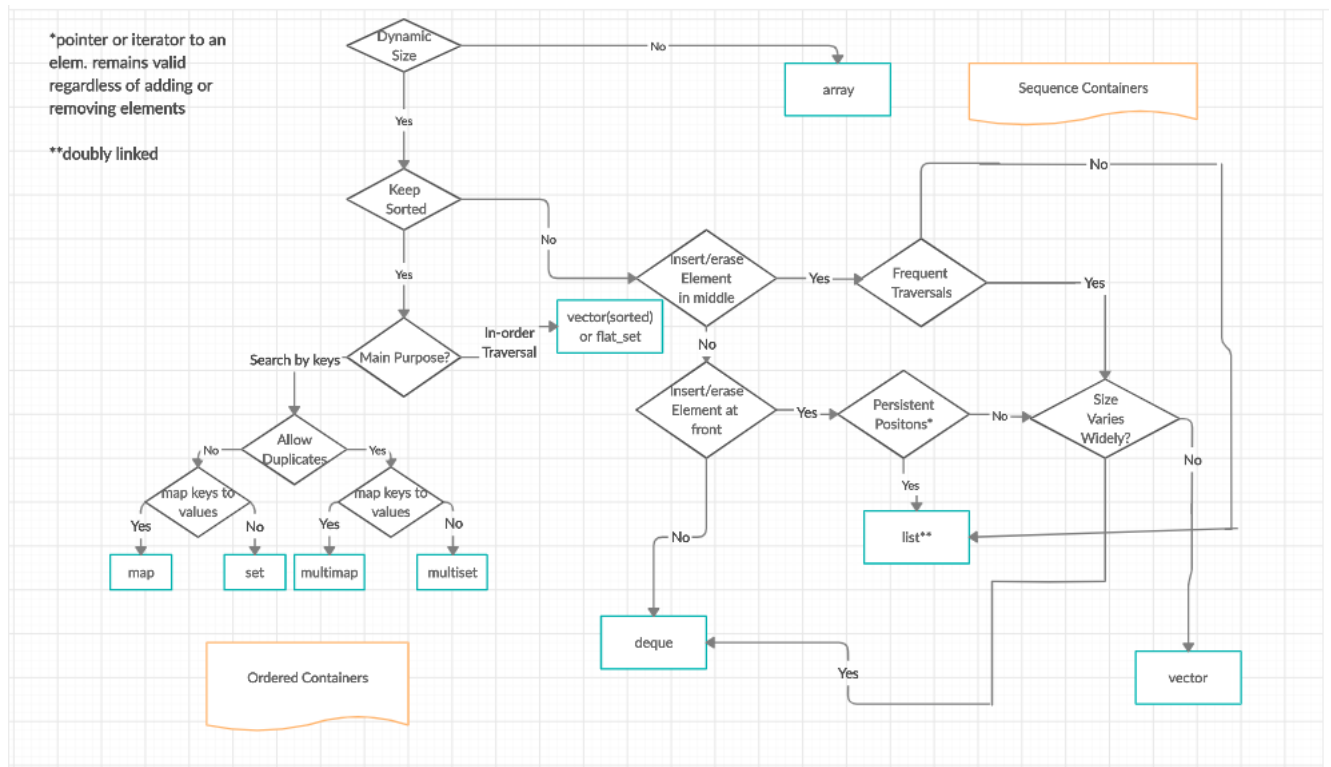


Figure 4.1: Decision Chart: Sequence And Ordered Containers

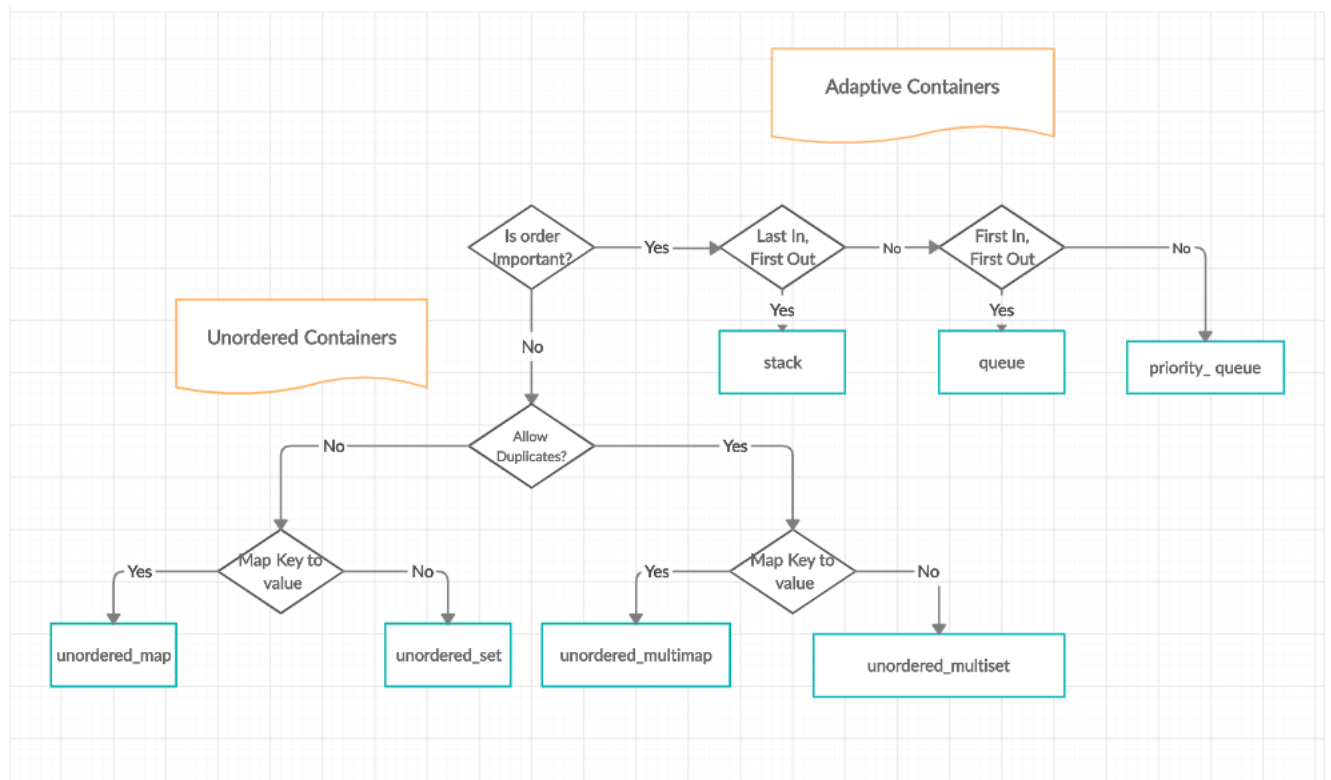


Figure 4.2: Decision Chart: Adaptive And Unordered Containers

1 Links to the CPP Reference documentation